

NATIONAL ECONOMICS UNIVERSITY – SCHOOL OF TECHNOLOGY

FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

COURSE HANDBOOK &
SELF-STUDY GUIDE

DSAI1003: FUNDAMENTAL PROGRAMMING CONCEPTS IN
PYTHON

Fundamental Programming Concepts in Python

Instructor:	Dr. Vu Duc Minh	Program:	Data Science in Finance & E-commerce (DSFE)
Course Code:	DSAI1003	Workload:	3 Credits (45h Lectures + 22.5h Lab + 90h Self-study)
Decision:	975/QĐ-ĐHKTQD (30/03/2024)	Course Type:	Compulsory Core Foundation
Website: dmvn.github.io/Fundamental-of-Programming-Concepts-in-Python			

Contents

1	Course Overview & Detailed Syllabus	1
1.1	Course Description & Pedagogical Philosophy	2
1.2	Course Objectives (COs) & Learning Outcomes (CLOs)	2
1.3	Textbooks & Learning Resources	2

1.4	Assessment Scheme & Course Regulations	2
1.5	15-Week Detailed Course Schedule	3
2	Effective Python Self-Study Methodology	4
2.1	The Nature of Learning Programming & 4 Fatal Pitfalls	4
2.2	The 9-Step Active Study Cycle	4
2.3	Breakthrough Active Learning Techniques	5
2.3.1	The Predict → Run → Explain Technique	5
2.3.2	The Teach-Back Technique	5
2.3.3	Active Recall & Spaced Repetition	5
2.4	The 5-Level Learning Ladder	5
2.5	Standard Debugging Checklist & Error Categories	6
2.6	Building a Personal Learning Log & Error Log	6
3	Strategic Use of Artificial Intelligence (AI) in Learning	7
3.1	Accurate Positioning of AI in Programming Education	7
3.2	The 4-Part Structured Prompt Framework	7
3.3	The 5T Interaction Principle	7
3.4	The Three-Level Hint System	8
3.5	Actionable Prompt Toolkit	8
3.6	7-Point AI Verification Checklist	9
4	Week 1 Action Plan for Success	9

1. Course Overview & Detailed Syllabus

1.1 Course Description & Pedagogical Philosophy

The course **Fundamental Programming Concepts in Python (DSAI1003)** is an introductory prerequisite course in the Bachelor of Data Science in Finance and E-commerce (DSFE) program. It is designed to establish a solid foundation in algorithmic thinking, program structure, and computational problem-solving using Python.

Key Characteristics of the Course Approach

- **No prior programming experience required:** Students only need a solid grasp of high school algebra and fundamental logical thinking skills.
- **Practical real-world orientation:** Examples, coding exercises, and case studies are drawn directly from economics, corporate finance, retail transactions, and e-commerce.
- **Systematic pedagogical progression:** Students start from algorithmic design (pseudocode, flowcharts), master fundamental control structures, implement modular functions, leverage versatile built-in collections, and culminate in Object-Oriented Programming (OOP) fundamentals.

1.2 Course Objectives (COs) & Learning Outcomes (CLOs)

The course aims to fulfill 7 overarching objectives (**G1** through **G7**), mapped to measurable Program Learning Outcomes (PLOs):

1.3 Textbooks & Learning Resources

The curriculum integrates three internationally recognized textbooks:

1. **[1] Python for Everyone** (3rd Edition, 2019) – *Cay Horstmann, Rance Necaise*, John Wiley & Sons. (Main textbook for syntax, algorithmic patterns, and business applications).

No.	Course Objective Description (COs)	PLO	Level
G1	Implement simple console programs or graphical user-interface programs to solve business problems.	1.4, 2.2	4
G2	Use decision structures (<code>if-elif-else</code>) and loop structures (<code>while</code> , <code>for</code>) for repetitive tasks.	1.4, 2.2	4
G3	Design and implement modular functions and recursive functions for divide-and-conquer tasks.	1.4, 2.2	4
G4	Proficiently utilize built-in data collections (<code>List</code> , <code>Tuple</code> , <code>Set</code> , <code>Dictionary</code>) to process data.	1.4, 2.2	4
G5	Implement file I/O operations (<code>text</code> , <code>binary</code>) and handle exceptions safely (<code>try-except</code>).	1.4, 2.2	4
G6	Acquire and apply core Object-Oriented Programming (OOP: <code>Classes</code> , <code>Inheritance</code> , <code>Polymorphism</code>).	1.4, 2.2	3
G7	Develop self-study ability, independent problem-solving, teamwork, and personal accountability.	2.4, 3.1	3

Table 1: DSAI1003 Course Objectives Mapping

- [2] **Intro to Python for Computer Science and Data Science: Learning to Program with AI, Big Data and The Cloud** (2020) – *Paul Deitel, Harvey Deitel*, Pearson Education. (Main textbook for data processing mindset and modern AI applications).
- [3] **The Python Workbook: A Brief Introduction with Exercises and Solutions** (2nd Edition, 2019) – *Ben Stephenson*, Springer. (Self-study exercises with comprehensive solutions).

1.4 Assessment Scheme & Course Regulations

Academic performance is assessed through 3 independent components, ensuring continuous feedback and accurate measurement of practical programming competence:

Assessment Component	Scope & Content	Timing	Criteria & Tools	Weight
Homework (HW)	Theoretical questions & coding assignments	Bi-weekly (≥ 4 sets)	Algorithmic logic, code clarity, deadline compliance (LMS)	30%
Midterm Exam	Core concepts & algorithms (Lessons 1–6)	Week 7	Written test or lab computer exam	30%
Final Exam	Comprehensive practical programming (Lessons 1–10)	Week 15	Lab computer practical test (execution, OOP, error handling)	40%

Table 2: DSAI1003 Course Assessment Scheme

Important Course Regulations

- Attendance:** Students are required to attend a minimum of **80%** of scheduled class sessions.
- Passing requirement:** The final weighted course grade must be at least **5.0 / 10.0**.
- Classroom laptops:** Personal laptops may only be used during designated hands-on coding exercises under instructor guidance; unauthorized non-academic activities are strictly prohibited.

1.5 15-Week Detailed Course Schedule

The 15-week semester is organized into 2 distinct pedagogical phases:

Phase 1: Syntax Foundations, Control Structures & Algorithms (Weeks 1 – 7)

Phase 2: Advanced Collections, File I/O & Object-Oriented Programming (Weeks 8 – 15)

Week	Topic & Core Content	Reading	Activities & Assessment
1	Lesson 1: Introduction & Algorithms Computer architectures, Python interpreter, compilation vs. runtime errors, pseudocode, flowcharts.	[1] Ch. 1 [2] Ch. 1	Syllabus introduction, environment setup (Python 3.11+, VS Code, Google Colab).
2	Lesson 2: Software Process & Data Types Numbers, strings, arithmetic expressions, assignments, variables, built-in functions.	[1] Ch. 2 [2] Ch. 2, 4	Live coding, interactive Q&A. Assign Homework #1 .
3	Lesson 3: Decision Structures Statements <code>if</code> , <code>if-else</code> , <code>if-elif-else</code> , Boolean logic and truth tables.	[1] Ch. 3 [2] Ch. 3	Branching practice, branch coverage testing. Due: Homework #1.
4	Lesson 4: Loop Structures Loops <code>while</code> , <code>for</code> , nested iterations, string indexing and traversals.	[1] Ch. 4 [2] Ch. 3	Accumulator and sentinel loop patterns. Assign Homework #2 .
5	Lesson 5: Function Design Syntax <code>def</code> , parameters, return values, variable scope, recursion basics.	[1] Ch. 5 [2] Ch. 4	Modularization and divide-and-conquer. Due: Homework #2.
6	Lesson 6: Lists & Tuples Linear arrays, negative indexing, slicing, List methods, 2D nested tables.	[1] Ch. 6 [2] Ch. 5	Tabular data structures and operations. Assign Homework #3 .
7	Midterm Exam Comprehensive review & evaluation of Lessons 1 through 6.	[1] Lessons 1–6	Midterm Examination (Weight: 30%).

Table 3: Course Schedule: Phase 1 (Weeks 1 – 7)

Week	Topic & Core Content	Reading	Activities & Assessment
8–9	Lesson 7: File I/O & Exception Handling Text file reading/writing, <code>sys.argv</code> , defensive error handling (<code>try-except</code>).	[1] Ch. 7 [2] Ch. 4	Persistent data storage & robust software design. Assign Homework #4 .
10	Lesson 8: Sets & Dictionaries Hash tables, key-value mappings, set operations, nested JSON-like records.	[1] Ch. 8 [2] Ch. 6	Commercial transaction data processing & business analytics.
11–12	Lesson 9: Objects & Classes (OOP – Part 1) OOP philosophy, <code>class</code> declarations, <code>__init__</code> constructor, encapsulation, instance methods.	[1] Ch. 9 [2] Ch. 10	Modeling banking and invoice entities. In-class OOP mini-lab.
13–14	Lesson 10: Inheritance & Polymorphism (OOP – Part 2) Inheritance hierarchy, subclassing, <code>super()</code> , method overriding, polymorphism.	[1] Ch. 10 [2] Ch. 10	Advanced OOP software architecture. Final exam review guidelines.
15	Course Review & Final Exam Prep Comprehensive synthesis of DSAI1003, instructor Q&A session.	All resources	Computer lab mock test & hands-on skills consolidation.

Table 4: Course Schedule: Phase 2 (Weeks 8 – 15)

2. Effective Python Self-Study Methodology

2.1 The Nature of Learning Programming & 4 Fatal Pitfalls

Learning programming differs fundamentally from memorizing theoretical subjects or rote syntax memorization. **The essence of programming is systematic problem-solving: decomposing problems and translating algorithms into precise computer instructions.**

Core Self-Study Principle

"Learn programming by reading less, practicing more, and thinking before looking at solutions."

Novice programmers frequently stumble into four psychological pitfalls:

1. **The Illusion of Competence via Passive Reading (Reading without running):** Skimming sample code and thinking *"I understand this"*. When the screen is turned off, the learner cannot write a single line from scratch.
2. **Blind Copy-Pasting:** Code executes, but the learner cannot explain why specific statements exist or what the parameters signify.
3. **Checking Solutions Prematurely:** Giving up and opening the answer immediately upon encountering difficulty. This bypasses the critical mental struggle where genuine problem-solving capacity is built.
4. **Cramming Too Many Concepts Simultaneously:** Attempting to rush through Variables → Branching → Loops → Functions → OOP in two days without allowing time for concepts to consolidate through deliberate practice.

2.2 The 9-Step Active Study Cycle

To truly master a programming concept, students are strongly advised to follow this closed-loop learning cycle:

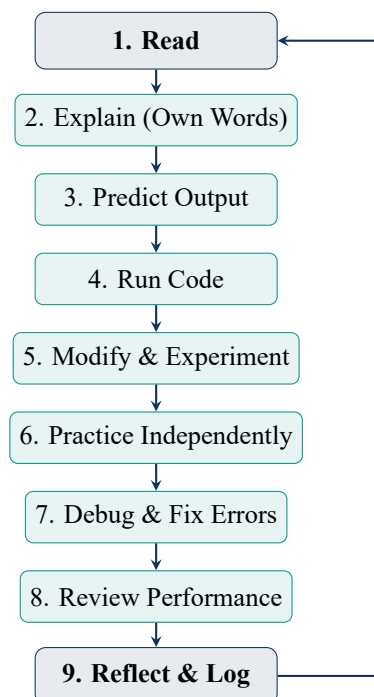


Figure 1: The 9-Step Active Programming Study Cycle

Self-study time should not be distributed equally across steps. **Over 70% of study time must be spent directly on: typing code, observing runtime behavior, analyzing tracebacks, and solving unassisted exercises (The 70/30 Rule).**

2.3 Breakthrough Active Learning Techniques

2.3.1 The Predict → Run → Explain Technique

Before clicking the Run button on your IDE, force yourself to write down on paper or predict mentally:

- Exactly what text or numbers will appear on the screen?
- How does each line change variable states and memory contents?

When actual execution differs from your prediction, that discrepancy marks the exact moment your mental model of Python's execution model is upgraded.

2.3.2 The Teach-Back Technique

Explain a concept (e.g., *"How does a function return value differ from a print statement?"*) to someone without a technical background, or record your explanation. If you find yourself saying: *"It runs like that because that is how it is"*, you have not yet mastered the underlying mechanism.

2.3.3 Active Recall & Spaced Repetition

Avoid rereading slides repeatedly. Close all materials and answer diagnostic questions: *What data type does input() return? How do you cast to an integer? What error occurs if you concatenate a string with an integer?* Schedule reviews after 1 day, 3 days, 7 days, and 14 days to transform transient memory into enduring intuition.

2.4 The 5-Level Learning Ladder

Whenever tackling a new programming topic, ascend systematically through these difficulty tiers:

- **Level 1 – Modify Existing Code:** Change variable values, convert integers to floats, observe corresponding output changes.
- **Level 2 – Reconstruct from Description:** Cover the solution, read the problem prompt, and write the complete program from scratch.
- **Level 3 – Combine Multiple Concepts:** For instance: Combine a `for` loop with an `if-else` branch to filter even and odd integers from a list.
- **Level 4 – Real-World Contextual Problems:** Compute progressive electricity tariffs, calculate personal income tax brackets, or model compound interest.
- **Level 5 – Mini Project:** Build a console-based order management system or an expense transaction analyzer.

2.5 Standard Debugging Checklist & Error Categories

Encountering errors is inevitable and an essential component of learning programming. When your program crashes, do not panic or randomly delete code. Apply this structured checklist:

Standard 8-Step Debugging Protocol

1. Read the entire traceback message from the bottom line upward.
2. Pinpoint the exact file and line number where the failure occurred.
3. Identify the specific error class: **SyntaxError**, **NameError**, **TypeError**, **IndexError**, **ZeroDivisionError**.
4. Insert temporary `print()` statements or use a debugger to inspect real-time variable values.
5. Verify variable types using `type()`.
6. Compare observed runtime output with theoretical expectations.
7. Test the program against the smallest, simplest possible input data.
8. Change only **one variable or line at a time** between test executions.

2.6 Building a Personal Learning Log & Error Log

Dedicate 5 minutes after every study session to update your personal log (`learning_log.md`):

Encountered Error	Root Cause	Resolution Strategy	Understood?
<code>TypeError: can only concatenate str to str</code>	<code>input()</code> always returns a string (<code>str</code>), which cannot be directly added to integers.	Cast with <code>int(input())</code> or <code>float(input())</code> before math operations.	Yes
<code>IndexError: list index out of range</code>	Iteration reached index N , while the valid list indices range from 0 to $N - 1$.	Adjust boundaries using <code>range(len(lst))</code> .	Yes
Infinite while loop	Failed to increment the loop counter inside the <code>while</code> body.	Append <code>i += 1</code> at the end of the loop body.	Yes

Table 5: Sample Personal Error Log

3. Strategic Use of Artificial Intelligence (AI) in Learning

3.1 Accurate Positioning of AI in Programming Education

Large Language Models (ChatGPT, Claude, Gemini) provide unprecedented educational leverage. However, their true pedagogical value depends entirely on **how learners interact with them**.

Hazards of AI Over-Reliance

If a student merely pastes problem statements: *"Solve this problem for me"* and copies the generated code:

- The student never grasps algorithmic construction or architectural design rationales.
- In proctored exams without AI access, the student is left entirely paralyzed.
- Over time, analytical reasoning and critical problem-solving capabilities suffer severe cognitive atrophy.

Positioning AI: Personal Tutor & Coach

Treat AI as a dedicated **Personal Tutor and Coach**:

- Ask AI to **explain** difficult concepts using relatable analogies.
- Request **incremental hints** when stuck, rather than full solutions.
- Ask AI to **generate diagnostic quizzes** to assess your conceptual grasp.
- Submit your own hand-written code for **code review and optimization suggestions**.

3.2 The 4-Part Structured Prompt Framework

To guide AI toward educational coaching rather than simply giving away answers, an effective prompt must embody four key components:

1. Context	Who are you, what level are you at?	<i>"I am a first-year student beginning foundational Python programming..."</i>
2. Problem	Where precisely are you stuck?	<i>"I struggle to differentiate between variable assignment and equality comparison..."</i>
3. Task	What specific assistance do you need?	<i>"Explain this with an everyday analogy and a short code snippet under 8 lines..."</i>
4. Constraint	What boundaries must AI respect?	<i>"Do not use advanced features like functions or classes; ask me 2 check questions without providing answers yet."</i>

3.3 The 5T Interaction Principle

A rigorous workflow between student and AI is captured by the **5T Principle**:



Figure 2: The 5T Principle for Effective AI Interaction

1. **Think:** Identify Inputs, Outputs, and algorithmic strategies before opening an AI chat.
2. **Try:** Type an initial draft independently, even if incomplete or buggy.

3. **Talk:** Query AI specifically on the pinpointed bottleneck; share existing code, observed errors, and expectations.
4. **Test:** Never accept AI code on faith. Copy it into your IDE, run it, and stress-test it against edge cases.
5. **Teach-back:** Close the chat window, articulate the mechanism in your own words, and record the lesson in your notebook.

3.4 The Three-Level Hint System

When confronting challenging assignments, train your AI assistant to dispense guidance in three calibrated tiers:

The 3-Level Scaffolding Framework

- **Level 1: Idea Hint:** AI outlines the mathematical logic or algorithmic pattern without providing any code.
- **Level 2: Structure Hint:** AI indicates appropriate programming constructs (e.g., recommend a for loop nested with an if condition using the modulo operator %).
- **Level 3: Scaffold Hint:** AI provides a skeletal code template with blank placeholders (# TODO) so the student must implement the critical logic.

3.5 Actionable Prompt Toolkit

The following high-leverage prompts can be customized and utilized directly during self-study:

Actionable Prompt Template: Explaining a New Concept (Concept Prompt)

I am a beginner learning Python. Please explain the concept of [CONCEPT NAME, e.g., Recursive Functions]. Requirements:

1. Explain the underlying intuition using simple language and an everyday analogy.
2. Provide a clean, annotated Python code example under 10 lines.
3. Point out 1 common pitfall or misconception that beginners frequently make.
4. Formulate 2 short diagnostic questions to test my understanding; do not provide answers yet.

Actionable Prompt Template: Requesting Problem Hints (Socratic Hint)

I am working independently on this exercise: "[EXERCISE DESCRIPTION]"
 My current idea and code attempt:
 [PASTE YOUR CODE ATTEMPT HERE]
 I am currently blocked at step: [DESCRIBE BOTTLENECK].
 Requirement: DO NOT write the complete solution. Provide a LEVEL 1 HINT (conceptual or algorithmic suggestion) so I can continue thinking and coding on my own.

Actionable Prompt Template: Debugging Assistance (Root Cause Discovery)

I wrote the following Python code to achieve: [DESCRIBE INTENDED BEHAVIOR].
 My current code:
 [PASTE CURRENT CODE HERE]
 When executed, I encounter this error/unexpected output: [PASTE TRACEBACK OR OUTPUT].
 Requirements:

1. Explain the root cause of this error.
2. Identify the specific lines that require attention.
3. Ask me a guiding question so I can correct the bug myself; DO NOT rewrite the code for me.

Actionable Prompt Template: Code Review and Optimization (Pythonic Style)

I have completed this programming problem and the code runs correctly:

[PASTE COMPLETE WORKING CODE HERE]

Please act as an experienced Python Instructor and Software Engineer:

1. Evaluate code quality: variable naming conventions, readability, and structural modularity.
2. Does this implementation handle potential edge cases safely?
3. Suggest 1 concrete optimization to make the code more concise and idiomatic (Pythonic), explaining the rationale.

Actionable Prompt Template: Self-Quiz Generation

I have just completed studying these topics: [LIST TOPICS, e.g., Lists, Tuples, and Slicing].

Please generate a 6-question diagnostic quiz for me:

- 2 Multiple-Choice questions (4 options A, B, C, D; exactly 1 correct answer).
- 2 Predict-the-Output questions (short code snippets under 5 lines).
- 2 Find-the-Bug questions (short snippets containing exactly 1 bug to fix).

Constraint: Keep questions within beginner scope. DO NOT show answers yet; wait for my responses before grading and providing explanations.

3.6 7-Point AI Verification Checklist

Before accepting any AI-generated code or explanation, students must cross-examine it against this 7-point checklist:

- ☐ **1. Curriculum Compatibility:** Does the code avoid unintroduced external libraries or advanced concepts beyond the current lesson?
- ☐ **2. Practical Execution:** Have you copied the code into your IDE and verified clean execution without warnings or errors?
- ☐ **3. Exact Specification Match:** Does the terminal output match assignment requirements down to formatting and casing?
- ☐ **4. Boundary Testing:** Does the program function properly with inputs like 0, negative values, empty strings, or empty lists?
- ☐ **5. Standard Terminology:** Is the terminology consistent with the course's official textbook (*Python for Everyone*)?
- ☐ **6. Line-by-Line Explanation:** Can you point to every single line and explain precisely why it is there?
- ☐ **7. Independent Reproduction:** If the AI chat was deleted, could you rewrite the program independently within 10 minutes?

The Inviolable Golden Rule

**“NEVER SUBMIT OR PRESERVE SOURCE CODE
THAT YOU CANNOT THOROUGHLY EXPLAIN YOURSELF.”**

4. Week 1 Action Plan for Success

To attain top academic honors (Grade A/A+) in DSAI1003, students should execute these concrete action items during the first week:

1. **Set up your computing environment:**

- Install the latest stable Python version (Python 3.11+) from <https://www.python.org/>.
- Install **Visual Studio Code (VS Code)** with the official *Python Extension Pack*.
- Sign in to **Google Colab** to enable seamless interactive programming from any browser.

2. Access and organize learning resources:

- Bookmark the official course GitHub repository: <https://github.com/vudmvn/Fundamental-of-Programming-Concepts-in-Python>.
- Read the detailed syllabus (`syllabus-en.md`) and Chapter 1 of *Python for Everyone*.

3. Establish your personal self-study system:

- Create a dedicated study directory named `DSAI1003_Python/` on your laptop or Google Drive.
- Create a `learning_log.md` file to document lessons, errors, and key insights weekly.
- Save the Prompt Toolkit into your notes app to facilitate thoughtful, disciplined AI interactions.

WISHING ALL DSFE STUDENTS

**AN INSPIRING SEMESTER, MASTERING COMPUTATIONAL THINKING
& HARNESSING MODERN AI TO ELEVATE YOUR PROFESSIONAL POTENTIAL!**

Faculty of Data Science and Artificial Intelligence – School of Technology
National Economics University (NEU)
Hanoi, Academic Year 2024 – 2025